12.4. Huffman Coding

Compress more by storing less.

Previous Sections:

- 12.1. Heaps
- 12.2. Operations on Heaps
- 12.3. Heap Sort

Contents:

Another great application of heaps is **Huffman Coding**.

Huffman Coding is a lossless data compression algorithm that reduces the number of bits required to store or transmit data. The main idea is simple: characters that appear frequently are assigned shorter codes, while characters that appear less frequently are assigned longer codes.

This technique is widely used in file compression, image formats, and communication systems because it can significantly reduce the amount of data that needs to be stored or transmitted.

1. What is Huffman Coding?

Before understanding Huffman Coding, let's first understand the idea of **data compression**.

Consider the message: `Hellooo!`

Using ASCII encoding, every character requires **8 bits**.

Character	ASCII Code
H	01001000
e	01100101
l	01101100
l	01101100
o	01101111
o	01101111
o	01101111
!	00100001

Since there are 8 characters: `8 characters × 8 bits = 64 bits` . So the message requires **64 bits** to store.

Notice something interesting. The message contains only **5 distinct characters**: `H, e, l, o, !` . To distinguish between 5 different symbols, we do not necessarily need 8 bits. Since:

- 2 bits → 4 symbols (not enough)
- 3 bits → 8 symbols (enough)

We can create a custom encoding table.

Character	Encoding
H	000
e	001
l	010
o	011
!	100

Now the message becomes:

```
H    e    l    l    o    o    o    !
000 001  010  010  011  011  011  100
```

Visualizing the encoded stream:

```
000|001|010|010|011|011|011|100
```

Total bits used: `8 characters × 3 bits = 24 bits`. Instead of 64 bits (ASCII), we now use 24 bits. This saves: $64 - 24 = 40$ bits or approximately 62.5% reduction

1.1. Fixed-Length Encoding

The encoding above is called **Fixed-Length Encoding** because every character uses the same number of bits.

```
H → 000
e → 001
l → 010
o → 011
! → 100
```

All symbols use exactly **3 bits**. Visual representation:

```
Character : H   e   l   o   !
Code      :000 001 010 011 100
Length    : 3   3   3   3   3
```

While this is already better than ASCII, it still does not take advantage of the fact that some characters appear more often than others.

In our message:

```
H → 1 time
e → 1 time
l → 2 times
o → 3 times
! → 1 time
```

Notice that `o` appears much more frequently than the other characters. Wouldn't it be better if `o` → very short code and `H, e, !` → longer codes since they rarely appear?

This is exactly the idea behind **Huffman Coding**.

1.2. Frequency-Based Encoding

Instead of giving every character the same code length, Huffman Coding assigns: **High frequency character → Short code; Low frequency character → Long code**

For example:

```
o → 0
l → 10
H → 110
e → 1110
! → 1111
```

Now the encoded message becomes:

```
H     e     l    l    o   o   o     !
110 | 1110 | 10 | 10 | 0 | 0 | 0 | 1111
```

Bit count:

```
H  = 3 bits
e  = 4 bits
l  = 2 × 2 = 4 bits
o  = 3 × 1 = 3 bits
```

```
!  = 4 bits

Total = 18 bits
```

Compared to:

```
ASCII           = 64 bits
Fixed-Length    = 24 bits
Huffman Coding  = 18 bits
```

Huffman Coding achieves an even better compression ratio because frequently occurring characters are represented using fewer bits.

The challenge is determining **which character should receive which code** while ensuring the encoded message can still be decoded correctly. To solve this problem efficiently, Huffman Coding uses a special binary tree called the **Huffman Tree**, which is constructed using a **Min Heap**.

2. Prefix Code in Huffman Coding

Before building a Huffman Tree, we must first understand an important concept called a **Prefix Code**. A prefix code is a set of binary codes where **no code is the prefix of another code**.

In other words, when reading a sequence of bits, we can immediately determine where one character ends and the next begins without needing any special separator.

2.1. What is a Prefix Code?

Consider the following set of codes: `A = {0, 10, 110, 111}`

This is a prefix code because:

- 0 is not the beginning of 10, 110, or 111
- 10 is not the beginning of 110 or 111
- 110 is not the beginning of 111

No code starts with another code. Therefore, the encoded message can always be decoded uniquely.

Now consider another set: `B = {0, 10, 110, 101}` . This is **not** a prefix code.

Notice: 10 and 101. The code **101** begins with **10**. Suppose we receive: 101, which should it be interpreted as: `10 | 1` or `101`. The decoder cannot know for sure. This ambiguity makes decoding impossible.

2.2. Fixed-Length Encoding and Prefix Codes

In the previous section, we used a fixed-length encoding:

```
H → 000
e → 001
l → 010
o → 011
! → 100
```

Since every code uses exactly 3 bits, no code can be the prefix of another. Therefore, fixed-length encoding automatically satisfies the prefix-code property.

However, prefix codes are not limited to fixed-length encodings. Variable-length encodings can also be prefix codes as long as no code starts with another code. This idea is what makes Huffman Coding possible.

2.3. Prefix Code as a Binary Tree

A prefix code can be represented using a binary tree. The rules are: Left branch → 0; Right branch → 1

Each character is stored in a leaf node. The code of a character is obtained by tracing the path from the root to that leaf. For example:

```
          Root
         /    \
      0/       \1
      A         *
               / \
            0/   \1
            B     *
                 / \
```

```
                    0/   \1
                    C     D
```

The corresponding codes are:

```
A → 0
B → 10
C → 110
D → 111
```

Notice that no code is a prefix of another because every character appears only at a leaf node.

Why Variable-Length Encoding?

Consider our message: `Hellooo!`. Character frequencies:

```
H → 1
e → 1
l → 2
o → 3
! → 1
```

We can immediately see that some characters appear more often than others. If we assign shorter codes to frequent characters and longer codes to rare characters, we can reduce the total number of bits required.

Suppose we choose the following prefix code:

```
o → 0
l → 10
H → 110
! → 1110
e → 1111
```

The corresponding coding tree is:

```
                Root
               /    \
           o(0)      *
                    / \
                 l(10) *
                      / \
                   H(110) *
```

```
                              / \
                          !(1110) e(1111)
```

The encoded message becomes:

```
H     e     l    l    o   o   o    !
110 | 1111 | 10 | 10 | 0 | 0 | 0 | 1110
```

Total bits used:

```
H  = 3 bits
e  = 4 bits
l  = 2 × 2 = 4 bits
o  = 3 × 1 = 3 bits
!  = 4 bits

Total = 18 bits
```

Compared to:

- ASCII Encoding = 64 bits
- Fixed-Length Coding = 24 bits
- Huffman Coding = 18 bits

The compression is significantly better. The challenge now becomes: ***How do we systematically generate the best prefix code?*** The answer is the **Huffman Tree Algorithm**.

3. Building the Huffman Tree

The Huffman Coding algorithm consists of three major steps:

- **Step 1: Count Character Frequencies**

Count how many times each character appears in the input string.

For the message: `Hellooo!` , we obtain:

```
o → 3
l → 2
H → 1
e → 1
! → 1
```

- **Step 2: Build the Huffman Tree**

The Huffman Tree is built from the bottom upward.

Step 2a: Create Leaf Nodes

Create one leaf node for each character. The node weight equals the character frequency.

```
H(1)   e(1)   !(1)   l(2)   o(3)
```

Step 2b and 2c: Merge the Two Smallest Nodes

Select the two nodes with the smallest frequencies: `H(1)` and `e(1)`

Create a new parent node:

```
    (2)
   /   \
H(1) e(1)
```

Push the new node back into the list.

```
!(1)   l(2)   (H,e)(2)   o(3)
```

Repeat the process → Select: `!(1)` and `l(2)`

Create:

```
     (3)
    /   \
!(1)  l(2)
```

New list:

```
(H,e)(2)   o(3)   (!,l)(3)
```

Repeat again → Select: `(H,e)(2)` and `o(3)`

Create:

```
        (5)
       /   \
    (2)    o(3)
   /   \
H(1) e(1)
```

New list:

```
(!,l)(3)   (H,e,o)(5)
```

Repeat one final time → Select: `(!,l)(3)` and `(H,e,o)(5)`

Create:

```
              (8)
            /     \
         (3)      (5)
        /  \     /  \
!(1) l(2) (2)  o(3)
           / \
        H(1)e(1)
```

Only one node remains. This node becomes the root of the Huffman Tree.

Step 2d and 2e: Huffman Tree Completed

Final Huffman Tree:

```
              (8)
            /     \
         (3)      (5)
        /  \     /  \
!(1) l(2) (2)  o(3)
           / \
        H(1)e(1)
```

- **Step 3: Generate Huffman Codes**

Assign: Left branch → 0; Right branch → 1. Traversing from the root to each leaf:

```
! → 00
l → 01
H → 100
e → 101
o → 11
```

(Actual codes may vary depending on how equal-frequency nodes are arranged.)

These generated codes are Huffman Codes and can be used to compress the original message efficiently.

The use of a Huffman Tree guarantees that the resulting encoding is a valid prefix code while minimizing the total number of bits required to represent the input data.

4. Huffman Decoding

Huffman Decoding is simply the reverse process of Huffman Coding.

While Huffman Coding converts a string into a compressed binary sequence, Huffman Decoding reconstructs the original string from the encoded bit stream.

To decode a Huffman-encoded message, we must have the corresponding **Huffman Tree** used during the encoding process. The decoder uses this tree to determine which character is represented by each sequence of bits.

How Huffman Decoding Works

Recall the rule used when constructing the Huffman Tree: ***Left branch → 0; Right branch → 1***

To decode a binary sequence, we traverse the Huffman Tree according to each bit encountered. The decoding process consists of the following steps:

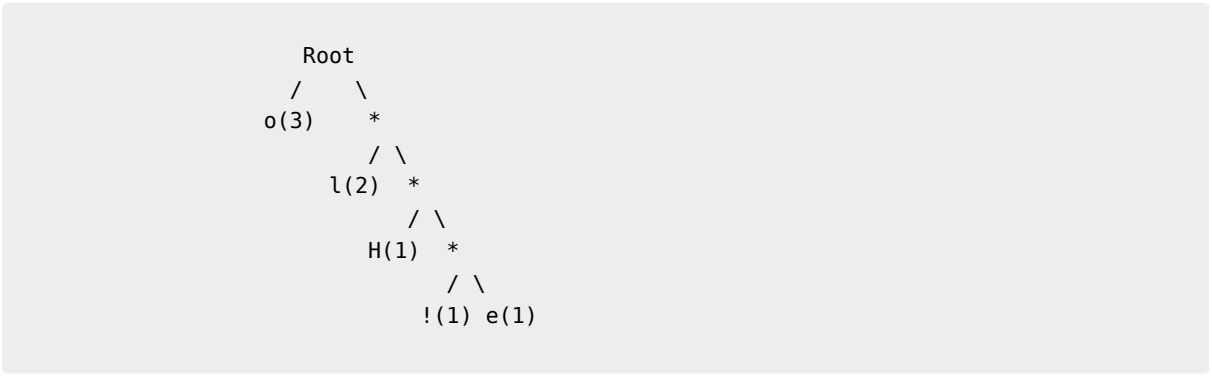
- Step 1: Start at the root node of the Huffman Tree.
- Step 2: Read the next bit from the encoded bit stream. If the bit is:
  - 0 → Move to the left child
  - 1 → Move to the right child

- **Step 3:** Continue traversing the tree until a leaf node is reached.
- **Step 4:** When a leaf node is encountered:
  - Output the character stored in the leaf node.
  - Return to the root node.
  - Continue decoding the remaining bits.

This process repeats until all bits in the encoded message have been processed.

Huffman Tree of `"Hellooo!"`

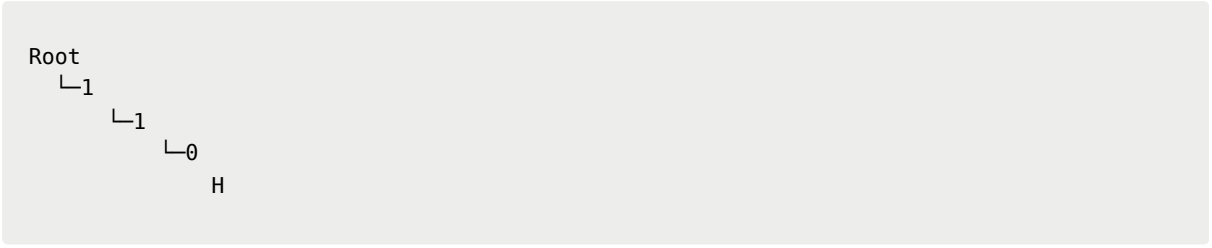
Using the Huffman Tree constructed in the previous section:



The corresponding codes are:

```
o → 0
l → 10
H → 110
! → 1110
e → 1111
```

Notice that each code can be obtained by tracing a path from the root node to its leaf node. For example:



Therefore: `H → 110` . Similarly:

```
e → 1111
l → 10
o → 0
! → 1110
```

Thus, the Huffman dictionary becomes:

```
{
    H : 110,
    e : 1111,
    l : 10,
    o : 0,
    ! : 1110
}
```

Encoding "Hellooo!"

Using the Huffman codes above:

```
H → 110
e → 1111
l → 10
l → 10
o → 0
o → 0
o → 0
! → 1110
```

Combining them together:

```
110 | 1111 | 10 | 10 | 0 | 0 | 0 | 1110
```

Removing the separators gives: `110111110100001110`

```
110111110100001110
```

This is the Huffman-encoded representation of the string: `Hellooo!`

Decoding Example

Suppose we receive the binary sequence: `1101111101000011110` . We start at the root node and read bits one by one.

- Decoding the first character - Read:
  - 1 → right
    1 → right
    0 → left
  - We arrive at: `H` → Output: `H`

Return to the root.

- Decoding the second character - Read:
  - 1 → right
    1 → right
    1 → right
    1 → right
  - We arrive at: `e` → Output: `e`

Return to the root.

- Decoding the third character - Read:
  - 1 → right
    0 → left
  - We arrive at: `l` → Output: `l`

Return to the root.

- Repeating the same process for the remaining bits produces:

```
H
e
l
l
o
o
o
!
```

Therefore: `1101111101000011110 → Hellooo!`

Multiple Valid Huffman Trees

An important observation is that Huffman Coding does not always produce a unique tree. During construction, we repeatedly select the two nodes with the smallest frequencies.

However, there may be multiple nodes sharing the same minimum weight. For example:

```
H(1)
e(1)
!(1)
```

All three nodes have the same frequency. Different pairs may be selected first: `(H, e); (H, !); (e, !)`

Each choice produces a slightly different Huffman Tree. As a result, the Huffman codes may differ. The encoded bit sequence may differ.

However, the compression efficiency remains the same. The resulting codes are still valid prefix codes. The message can still be decoded correctly using the corresponding Huffman Tree.

Therefore, Huffman Coding may generate multiple valid encoding dictionaries for the same input data whenever several characters share identical frequencies.

4. Implementing Huffman Coding

The Huffman Coding algorithm can be efficiently implemented using a **Min Heap**. The heap allows us to repeatedly retrieve the two nodes with the smallest frequencies, which is the core operation required when constructing a Huffman Tree.

The following implementation reuses the heap structure introduced in previous sections. Since our existing implementation is a Max Heap, it is converted into a Min Heap before constructing the Huffman Tree.

Use this code for the heap class

heap.py

4.1. Node Class

Each node in the Huffman Tree stores a character, its frequency, and references to its left and right child nodes.

```
# Huffman Coding with Heaps

class Node:
    def __init__(self, char=None, freq=0):
        self.char = char      # Character for Huffman encodi
ng
        self.freq = freq      # Frequency of the character
        self.left = None      # Left child
        self.right = None     # Right child

    def __lt__(self, other):
        return self.freq < other.freq
```

The `__lt__()` method allows nodes to be compared based on their frequencies, which is necessary for heap operations.

4.2. Building the Huffman Tree

The Huffman Tree is built by repeatedly selecting the two nodes with the lowest frequencies and merging them into a new parent node.

```
# Function to build the Huffman Tree based on the character
frequencies

def build_huffman_tree(frequency):
    heap = MaxHeap(len(frequency))

    # Insert all character-frequency pairs into the heap
    for char, freq in frequency.items():
        heap.insert(Node(char, freq))
    # Convert Max Heap into Min Heap
```

```
    heap.convert_min_heap()

    # Continue until only one node remains
    while heap.last > 0:
        left = heap.get_peak()
        heap.delete()

        right = heap.get_peak()
        heap.delete()

        # Create a merged node
        merged = Node(freq=left.freq + right.freq)
        merged.left = left
        merged.right = right

        # Insert merged node back into heap
        heap.insert(merged)
        # Maintain Min Heap property
        heap.convert_min_heap()

    return heap.get_peak()
```

Suppose the character frequencies are:

```
H → 1
e → 1
! → 1
l → 2
o → 3
```

Initially, the heap contains:

```
[1] H
[1] e
[1] !
[2] l
[3] o
```

The algorithm repeatedly removes the two smallest nodes:

```
H(1) + e(1)  → 2
!(1) + l(2) → 3
2 + o(3)    → 5
3 + 5       → 8
```

The final node with weight 8 becomes the root of the Huffman Tree.

4.3. Generating Huffman Codes

Once the Huffman Tree has been built, the codes can be generated by traversing the tree recursively. The convention used is: Left branch → 0; Right branch → 1

```python
# Function to generate Huffman codes

def generate_codes(node, prefix, codebook):
    if node is None:
        return

    # Leaf node
    if node.char is not None:
        codebook[node.char] = prefix

    generate_codes(node.left, prefix + "0", codebook)
    generate_codes(node.right, prefix + "1", codebook)
```

For example, given the Huffman Tree:

```
          Root
         /    \
      o(3)    *
             / \
          l(2)  *
               / \
            H(1)  *
                 / \
              !(1) e(1)
```

The generated codes are:

```
o → 0
l → 10
H → 110
! → 1110
e → 1111
```

These codes are stored in a dictionary called the **codebook**.

4.4. Decoding an Encoded Message

The following function decodes a Huffman-encoded bit stream using the generated codebook.

```
# Function to decode an encoded message

def decode_message(encoded_message, codebook):
    # Reverse mapping:
    # Huffman code → character

    reverse_codebook = {v: k for k, v in codebook.items()}
    current_code = ""
    decoded_message = ""

    for bit in encoded_message:
        current_code += bit

        if current_code in reverse_codebook:
            decoded_message += reverse_codebook[current_cod
e]
            current_code = ""

    return decoded_message
```

Suppose the codebook is:

```
{
    'H': '110',
    'e': '1111',
    'l': '10',
```

```
    'o': '0',
    '!': '1110'
}
```

The encoded string: `1101111101000011110` is decoded as:

```
110   → H
1111  → e
10    → l
10    → l
0     → o
0     → o
0     → o
1110  → !
```

Result: `Hellooo!`

Main Program

The following example demonstrates the complete Huffman Coding process.

```
def main():
    text = "Hellooo!"

    # Count character frequencies
    frequency = {}

    for char in text:
        frequency[char] = frequency.get(char, 0) + 1

    # Build Huffman Tree
    root = build_huffman_tree(frequency)

    # Generate Huffman codes
    codebook = {}
    generate_codes(root, "", codebook)

    print("Huffman Codes:")
    for char, code in codebook.items():
        print(char, ":", code)

    # Encode the message
    encoded_message = ""

    for char in text:
        encoded_message += codebook[char]

    print("\nEncoded Message:")
    print(encoded_message)

    # Decode the message
    decoded_message = decode_message(encoded_message, codebo
ok)

    print("\nDecoded Message:")
    print(decoded_message)


main()
```

Huffman Codes:
o : 0
l : 10
H : 110
! : 1110
e : 1111

Encoded Message:
1101111110100001110

Decoded Message:
Hellooo!

Summary

Huffman Coding is a lossless data compression algorithm that assigns shorter binary codes to frequently occurring characters and longer binary codes to less frequent characters. By using a Huffman Tree and repeatedly merging the nodes with the lowest frequencies, the algorithm generates an optimal prefix code that minimizes the total number of bits required to represent the input data.

The Min Heap plays a crucial role in the construction of the Huffman Tree because it allows efficient retrieval of the two lowest-frequency nodes during each iteration. Once the tree is built, encoding is performed by traversing the tree to generate binary codes, while decoding is accomplished by interpreting the encoded bit stream using the same Huffman Tree or codebook.

Huffman Coding is widely used in data compression systems because it significantly reduces storage and transmission requirements while preserving the original information exactly.

References

https://drive.google.com/file/d/1VfNgqFPkvnDGZeVkN_QqlHjWCDQRapBM/view?usp=drive_link

https://www.geeksforgeeks.org/dsa/huffman-coding-greedy-algo-3/

https://www.programiz.com/dsa/huffman-coding

https://www.youtube.com/watch?v=co4_ahEDCho

Next Section:

13. Graphs